Comparing Agentic AI Frameworks for Natural Language Wayang Queries

Jonathan Vitting Nielsen

May 21, 2026

Abstract

Apache Wayang is a unified data processing framework that allows users to query data across multiple heterogeneous platforms through a single interface. Prior work by Udengaard demonstrated that both single- and multi-agent large language model (LLM) pipelines, built on the Model Context Protocol (MCP) server framework, can successfully translate natural language queries into executable Wayang plans. As such systems mature, the choice of agentic framework becomes increasingly important for long-term maintainability and extensibility.

This report investigates whether replacing MCP servers with LangGraph, a graph-based agent framework, yields measurable benefits. We refactor both of Udengaard's systems to use LangGraph while preserving all internal logic, and evaluate the two implementations across 1200 benchmark runs on a TPC-H dataset, using three queries of varying complexity, two agent configurations, and two reasoning effort levels.

Results show that LangGraph offers a statistically significant performance advantage for simple queries, where framework overhead is a meaningful fraction of total runtime. For medium and hard queries, LLM reasoning time dominates and the two frameworks perform comparably. Accuracy is not significantly affected by the framework change. I conclude that the primary benefit of LangGraph lies not in raw performance, but in its cleaner, more decoupled code structure, which I argue is better suited to future extension of these pipelines.

Contents

1 Introduction **1**

2 Background **3**

2.1 MCP servers 3

2.2 Previous Work 4

2.3 LangGraph 5

3 Solution **7**

3.1 Single-Agent System 7

3.2 Multi-Agent System 10

4 Evaluation **12**

4.1 Metrics 12

4.2 Methodology 12

4.2.1 Model, Dataset and Hardware 12

4.2.2 Queries 13

4.2.3 Variables 14

4.3 Results 14

4.3.1 Performance 14

4.3.2 Accuracy 17

4.3.3 Lines of Code 19

5 Related Work **20**

5.1 Universal Tool Calling Protocol (UTCP) 20

5.2 Microsoft Agent Framework (MAF) 20

6 Conclusion **21**

1 Introduction

Recent years have seen a large increase in raw data being generated across all industries, heightening the need for good data analytics in order for companies to remain competitive. Data analytics have, however, traditionally been confined to technically skilled workers such as engineers or programmers. Additionally, as data generation continues to rise, processing these larger amounts of data becomes an increasingly complicated procedure. It is not uncommon for data to be stored across several distinct platforms, including big-data systems designed to process large amounts of data efficiently. This forces developers to spend resources "gluing together" solutions, to make different data frameworks and platforms talk to each other.

Apache Wayang is a unified data processing framework, designed to eliminate this problem. Wayang integrates multiple different data processing systems under the hood, while providing a single, platform-agnostic, interface to users, where they can query their data [1]. While Wayang helps data workers by reducing the complexity of managing multiple different data processing platforms, the barrier of entry remains relatively high because coding and technical skills are still required to use Wayang effectively.

Concurrent with the large-scale data generation seen across the world, large language models (LLMs) have seen a meteoric rise in capabilities. One of the more recent developments in this area is agentic AI, where an LLM is given agency to perform actions in addition to outputting text [2]. These actions can be reading files, writing code or plotting graphs for the user. Further, multi-agent setups exist, where multiple specialized agents each work together to solve a task, like a team of human workers would. In theory, multi-agent processes should be able to reliably perform more complicated tasks than single-agent systems. However, multi-agent processes also increase the complexity of their setups significantly since each agent needs to be initialized individually, and agents additionally need to be able to communicate effectively with one another. A robust framework for managing agentic AI is thus essential in order to create the necessary infrastructure needed to scale up systems involving these agents.

Thanks to the recent improvements in LLM capabilities, we can try to use their reasoning power to enable non-technical users to use Wayang effectively by providing a natural language interface. In theory, a robust and reliable pipeline here would be able to translate natural language queries to Wayang-compliant syntax, and allow any worker, not just data technicians, access to efficient large-scale data analytics. However, this is not very straight

forward to do. In a previous work, researcher Alexander Udengaard successfully built and tested both single- and multi-agent systems for interpreting natural language queries into Wayang plans [3]. Both systems use the MCP server framework, however technology advances quickly in this space, and new frameworks for managing agentic AI are continuously emerging and maturing, possibly providing better performance or improved agent infrastructure. As Udengaard's systems for natural language interpretation is built upon, expanded and improved, a robust agentic AI framework is imperative to the overall longevity of the systems.

One such alternative to the MCP framework is LangGraph, a graph-based agent infrastructure framework, that is supposedly leaner and more decoupled than MCP, potentially making future improvements to the systems easier to implement. The overall goal of this report is to investigate whether we gain any benefits by changing the agent framework of Udengaard's original systems from MCP to LangGraph.

2 Background

This section presents the theoretical and technical context necessary to understand the comparison between MCP servers and LangGraph. In Section 2.1 I give a brief overview of MCP servers - note that these are presented in great detail in Udengaard's paper, and for more information I refer to that study. In Section 2.2 I describe Udengaard's single- and multi-agent systems for natural language processing into Wayang plans, and in Section 2.3 I present LangGraph and its technical details.

2.1 MCP servers

MCP servers are an open standard for integrating agentic behavior into LLMs introduced by engineers at Anthropic in 2024 [4]. MCP works on a client-server basis. An MCP client is typically an LLM chatbot, and the server provides the chatbot with a set of pre-defined tools that the chatbot can invoke. These are functions such as bash commands or selecting tables from a database. In fact, like functions or methods, tools can contain any type of logic we can think of. When an MCP client (again, typically an LLM chatbot) is first started, it loads a list of all available tools into memory. Each tool is fitted with a detailed natural language description that the LLM reads. The description is used to judge when it is appropriate to invoke the tool. When the tool is invoked, all code within that tool is executed, and we may optionally choose to return data back to the LLM if we need to. Note that the LLM does not have any insight into the logic of the tool beyond what is described to it in natural language, and potentially what is returned, meaning it cannot interact with the tool in any way beside invoking it and waiting for it to finish.

An example of an agent using an MCP server could be a meeting planner. The MCP server defines a tool that, when invoked, creates a meeting in the user's local calendar. The tool may return an error/success code to let the LLM know whether the meeting was successfully created. When the user is in a chatbot session and tells the MCP client to create a meeting the LLM will decide to invoke the tool, and a meeting is created. Connecting an LLM chatbot to an MCP server is the typical way of using MCP servers in a production setting, however for testing purposes we may also invoke the tools directly via a script. This is how I gathered the benchmark results for this report. The MCP protocol also supports other concepts besides tools, notably resources and prompts, however these will not be covered in this report.

The major benefit of the client-server architecture is interoperability. Any entity that can connect to the server via simple communication protocols can access the tools it provides. This is also the drawback: such entities do not need to be an LLM chatbot, but can be

adversaries with malicious intent. If they gain access, they can invoke the tools of the MCP server. Thus, care needs to be taken in defining the scope of the tools for the MCP server.

2.2 Previous Work

Udengaard's system for translation of natural language to Wayang plans uses MCP servers. Udengaard defines three tools for the LLM:

1. `query_wayang` 2. `load_schemas` 3. `get_wayang_results`

Loading the schemas provides schematic information on the dataset to the LLM and should only be done once whenever a new dataset is queried. `get_wayang_results` can be thought of as a simple getter-function that returns the results to the MCP-client. All of the pipeline's core logic lives inside the `query_wayang` tool.

When that tool is invoked in the single-agent pipeline, a Builder agent is first initialized with a detailed system prompt, that tells the agent its responsibilities. The Builder agent then takes the user query and tries to translate it to an intermediate JSON format that is easier for the LLM to parse, before a Plan Mapper script maps the intermediate format to a Wayang-compliant JSON syntax. After mapping, the plan is checked for common syntactical errors by a Plan Validator script, before being submitted to a Wayang server for execution. Note that the Mapper and Validator are both simple local scripts, and are not LLM-enhanced. If the plan fails at either validation or execution, and debugging is enabled, a Debugger agent will take the failed plan and try to improve it, before resubmitting it to Wayang. This happens iteratively until success or maximum iterations have been reached. When a plan is successfully executed it either writes the results to disk or gives the results back to the MCP client, depending on user preferences.

The multi-agent pipeline is a bit more involved. It defines the exact same tools as the single-agent pipeline, but several different agents each process the query before it is validated. The pipeline starts with a Specifier agent that rewrites the user query to be more specific, as well as a Selector agent that looks at the schemas of the dataset and extracts which tables it deems are relevant to the query. A Decomposer agent then splits the query into multiple substeps, where each substep is a set of logically adjacent Wayang operators. One substep could be mapping and reducing, while another could be to write to disk. A Builder agent is then loaded for each substep, and tries to generate the relevant Wayang operators from it. A Step Handler then merges all substeps, generating a full Wayang plan. A Refiner agent finally reviews the plan, and checks for consistency with the original query. The plan is finally forwarded to the same Mapper, Validator and Debugger steps as seen in the single-agent

system.

2.3 LangGraph

LangGraph is an open-source framework built and maintained by LangChain, initially released in 2023 [5]. LangGraph is a low-level extension of LangChain, a simpler agent framework. Their differences lie mostly in LangChain using direct sequential chains of nodes, whereas LangGraph is designed for more explicit user control and customization with branching graphs [6].

LangGraph models its workflow as a directed graph, where nodes are agents or functions, and edges are transitions. A node works like a function or method and may contain any type of programmatic logic we can think of. When the pipeline reaches a given node, the code within that node gets executed. In this paper I make a theoretical distinction between agentic nodes and functional nodes. Agentic nodes contain an LLM call, while functional nodes contain pure logic. An example could be a pipeline that collects data about different countries. Let's imagine a LangGraph graph that contain two nodes: one node calls an LLM for data about the population of a country, while the other node stores the data to disk. I call the first node an LLM node and the second a functional node. Figure 1 illustrates this supposed LangGraph graph. Note that the distinction between LLM and functional nodes is purely theoretical, and every node is programmatically defined the same way.

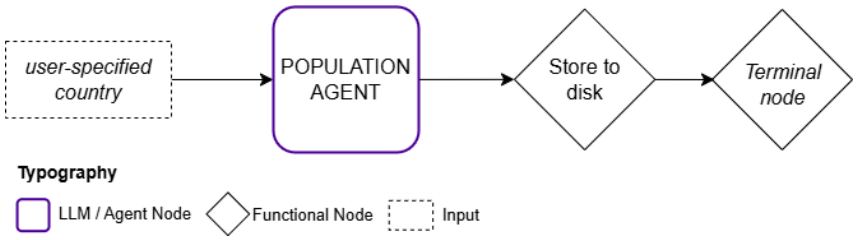


Figure 1: Example Country Population Graph

Edges work as transitions between nodes. When one node has finished executing its code, an edge decides which node the graph should executes next. There are two types of edges: normal and conditional. This distinction is functional, and the two types of edges are defined distinctly of each other. Normal edges simply route one node into another. Conditional edges route to any number of nodes based on which conditions are met. A conditional edge is simply a function that returns the name of the node to be executed, and may return the same node, creating a conditional loop. This means we can create branching if/else and looping sequences with relatively few lines of code.

Finally, LangGraph uses a data structure called a State. The State is a typed dictionary that persists across the lifespan of the graph that every node can access. The State may contain data that every node needs access to, like global variables, and may also be used to transfer data across nodes. Think back to the population data agent: When the first node queries an LLM for population data, that data gets stored in the graph State. The second node then reads those values from the State and saves them locally to disk. In addition to the population data, the State of this graph might also contain fields for the system prompt, user query, country, timestamps for logging, etc.

3 Solution

In this section I will present my effort to refactor the single- and multi-agent systems to use the LangGraph framework instead of MCP servers. In Section 3.1 I present the overall changes made to the single-agent system, and in Section 3.2 I detail changes to the multi-agent system. My overall goal is to recreate the single- and multi-agent pipelines using LangGraph instead of MCP servers, without changing any of the internal logic, if possible. Note that I will only recreate the `query_wayang` tool, as the other tools do not contain any critical logic. Links to GitHub repositories containing each LangGraph implementation can be found here: (Single-Agent) and (Multi-Agent).

3.1 Single-Agent System

Udengaard's systems both have a distinct graph-like structure to begin with, making the refactoring much easier. Figure 2 is taken from Udengaard's report [3, p. 14], and gives a graphical overview of his single-agent pipeline, that we see is clearly defined as a directed graph. In this system there are 3-4 steps:

1. A Builder agent
2. A Plan Mapper **(PM)**
3. A Plan Validator **(PV)**
4. An optional Debugger agent

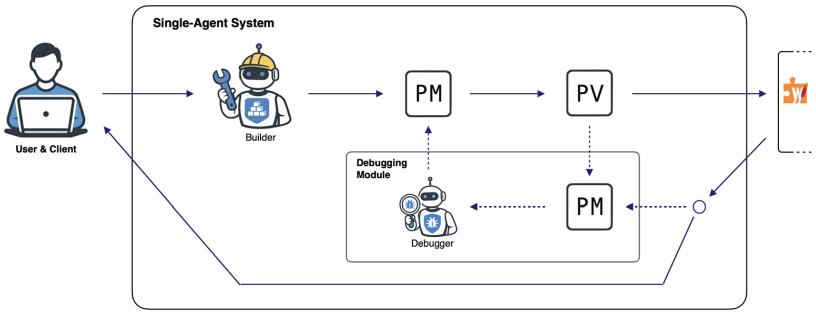


Figure 2: Single-Agent MCP Pipeline [3, p. 14]

Programmatically, each step is defined as its own class that is initialized whenever the MCP server is started. When `query_wayang` tool is invoked, the logic of step 1-3 (the Builder, Plan Mapper and Plan Validator) is sequentially run, meaning a plan is built, mapped, validated and sent for execution. If the debugger is active and a plan fails, the debugger runs iteratively until the plan executes properly or maximum iterations are reached.

Because of the graph-like structure of the MCP pipeline, converting the system to LangGraph was a straight-forward procedure: I simply encapsulate each step into its own node.

I create a Builder, Plan Mapper, Plan Validator, Executor, Debugger and Terminal node, with corresponding edges between them. The Executor sends the plan to the Wayang server for execution, and the Terminal node logs whether a plan was successfully executed or not. Figure 3 gives a graphical representation of the LangGraph implementation. While it may look more complicated than the MCP pipeline, the structure is overall quite similar, and much of the increased visual complexity comes from a lower level of abstraction.

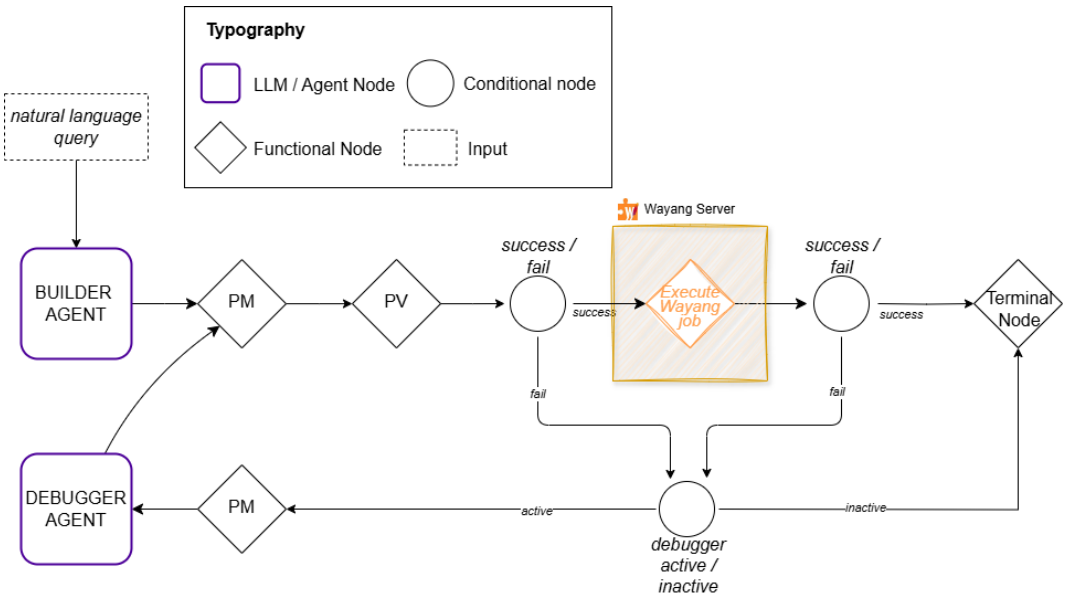


Figure 3: Single-Agent LangGraph Pipeline

Now that I have presented a graphical overview of the changes I believe it might be helpful to inspect an example node to see what the codebase looks like. We will use the Builder agent as an example. Figure 4 shows the single-agent MCP builder, and Figure 5 shows the corresponding Builder node in the LangGraph system.

```
### --- Generate Wayang Plan Draft --- ###

# Generate plan
print("[INFO] Generates raw plan")
response = builder_agent.generate_plan(describe_wayang_plan)
raw_plan = response.get("wayang_plan")

# Logging
print("[INFO] Draft generated")
logger.add_message("Agent Usage: BuilderAgent Information", {"model": str
(response["raw"].model), "usage": response["raw"].usage.model_dump()})
logger.add_message("Agent: BuilderAgent Raw Plan", raw_plan.model_dump())
```

Figure 4: Single-Agent MCP Builder Node Code Snippet

```python
def build_plan_node(state: WayangAgentState) -> dict:
    """Calls the Builder agent to generate an abstract WayangPlan from the query."""
    builder = Builder(model=state["model"], reasoning=state["reasoning"])
    response = builder.generate_plan(state["query"])

    _log(state, "Agent Usage: BuilderAgent Information", _get_raw_meta(response["raw"]))
    _log(state, "Agent: BuilderAgent Output", response["wayang_plan"].model_dump())

    return {"raw_plan": response["wayang_plan"]}
```

Figure 5: Single-Agent LG Builder Node Code Snippet

Similar to the original MCP code, the LangGraph node follows a quite simple structure. We define the LangGraph Builder node as a function that takes the graph State as an argument. We then initialize the Builder class with arguments gathered from the graph State, in this case model type and reasoning effort. We then extract the query from the State, and provide it to the Builder agent and instruct it to generate a plan. Finally, we feed the newly generated raw plan back into the graph State as a return value.

All of the nodes in the LangGraph pipeline follow this similar overall structure: Every node in the pipeline takes the graph State as an argument, initializes the relevant class, executes its logic, logs information and concludes by returning an update to the State. The pipeline is finished when it reaches the terminal node. Figure 11 in the Appendix shows how the graph is defined programmatically for the single-agent system, though it can also be seen in the GitHub repository.

As I have touched upon previously, every piece of information that the system needs is stored in the State, and viewing the State of a system can give a good overview of that system's dataflow. Figure 12 in the Appendix shows how the State is defined for the single-agent system, though it can likewise also be seen in the repository. This State contains the query, model type, reasoning level and whether the Debugger is active or not. Additionally, it stores intermediate output of each node (the raw plan, the Wayang plan), in addition to status codes and any potential raw response from the Wayang server, which are used for logging purposes. Finally, we also store an instance of the Debugger and Logger class, as these needs to have a consistent lifespan across several different nodes.

Overall, the LangGraph codebase is quite similar to the MCP implementation. The code, however, is arguably more explicit and readable because logic is encapsulated within multiple separate nodes, instead of one massive 200-line try-block within the `query_wayang` tool. Data is also explicitly stored in the State, instead of global or local variables littered across the tool.

3.2 Multi-Agent System

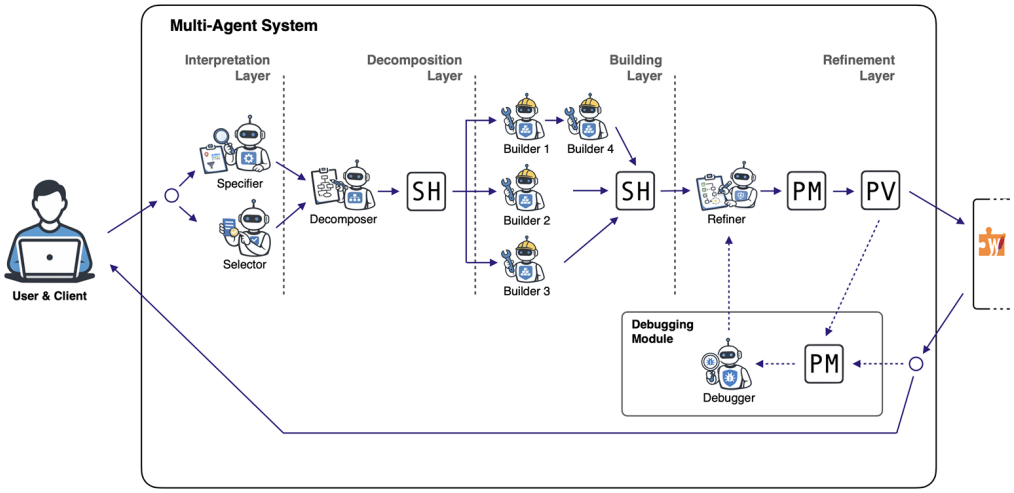


Figure 6: Multi-Agent MCP Pipeline [3, p. 18]

Although the multi-agent system is more complicated than its single-agent counterpart, the overall structure of the system follows the same basic principles outlined in the previous section. Figure 6 gives a graphical overview of Udengaard's MCP multi-agent pipeline [3, p. 18]. Overall there are 8-9 steps:

1. A Specifier agent
2. A Selector agent
3. A Decomposer agent
4. A Step Handler **(SH)**
5. A Builder agent
6. A Refiner agent
7. A Plan Mapper **(PM)**
8. A Plan Validator **(PV)**
9. An optional Debugger agent

For the LangGraph conversion, I once again encapsulate all the steps into their own nodes. The result is a Specifier, Selector, Decomposer, Builder, Refiner, Map Planner, Map Validator, Executor, Debugger and Terminal node, with corresponding edges between them. Note that while the Specifier and Selector node currently run sequentially in both MCP and LangGraph, in theory the LangGraph framework allows for those nodes to run in parallel, as they do not impact any data. Implementing this should produce slightly improved runtime.

Figure 7 shows a graphical representation of the LangGraph implementation. Note the similarities with the single-agent system. Most of the changes are in the initial stage, where we now use multiple different LLM nodes to process the query, before validation.

One of the main differences between the single- and multi-agent systems lies in the Builder node. The original MCP graphical representation in Figure 6 implies that multiple different agents are initialized and able to work on subplans in parallel. In reality, subplans are

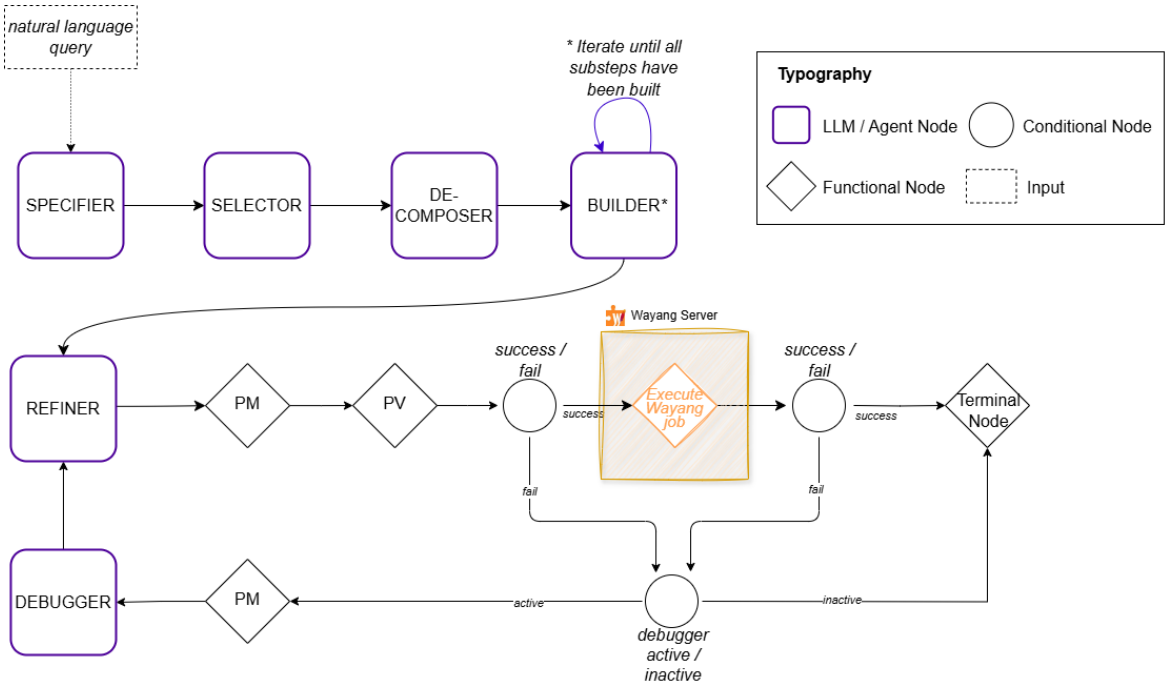


Figure 7: Multi-Agent LangGraph Pipeline

sequentially reliant, and are not able to be parallelized. In Udengaard's MCP implementation, the Decomposer and Step Handler generate a queue of subplans, and a Builder agent is initialized to translate the first subplan into Wayang operators. For each subsequent subplan in the queue, a new Builder agent is initialized, and is given the same system prompt along with the intermediate Wayang plan that has been built so far. When all steps in the queue are finished, the Wayang plan is complete.

The LangGraph Builder node follows this structure quite closely. The Step Handler is integrated into the `build_steps_node`, where it initially is responsible for building the queue. When the queue has been created, the Builder agent iterates through all the steps in a loop, before the step handler finally merges all the substeps into a raw plan, that is returned to the State.

Overall, as complexity increases so does the need for clarity in the codebase, and the multi-agent system is set up to benefit from the cleaner code more so than the single-agent system. In the next section I will compare results, including amounts of lines of code, to see if any measurable improvements have been gained by converting the system to LangGraphs.

4 Evaluation

This section outlines my results. I first define what kind of metrics I use for comparison in Section 4.1. In Section 4.2 I detail my methodology, including which queries, LLM model and dataset I use, and which variables I test for. In Section 4.3 I then present my overall results.

4.1 Metrics

To compare the MCP framework with the LangGraph framework, I first define a set of metrics that constitute a relevant comparison:

Performance: How long the system takes to execute a query. For MCP this is measured from the moment the `query_wayang` tool is invoked to the moment the tool is finished. For LangGraph this is measured from the moment the graph is invoked to the moment the graph executes its Terminal node. Both correspond to the moment results are retrieved from the Wayang server. By only measuring the time the pipeline takes to execute we isolate the performance away from external factors such as time spent in a chatbot or accidental tool-calls to `load_schemas`, and provide a fairer comparison between the two frameworks.

Accuracy: How many failed runs the system produces. Note that the testing script only marks a run as a failure if the Wayang plan was unsuccessfully executed, meaning a successful Wayang plan with an incorrect or partially incorrect result will still be logged as a success. Accuracy should in theory not be affected by the framework change, as LLM reasoning, not framework type, is chiefly responsible for producing a successful Wayang plan, however we still consider this metric to see if any surprising results arise.

Lines of Code: How many lines of code the pipeline occupies. For the MCP system I measure the amount of lines in the `query_wayang` tool, and for the LangGraph system I measure the amount of lines in the graph and State files. This comparison will give a rough idea of whether the new framework has made the codebase more or less verbose.

4.2 Methodology

4.2.1 Model, Dataset and Hardware

Model: I use OpenAI's ChatGPT-5.4 nano model for all the tests. This is the smallest of the GPT-5 family. Larger models produce successful results more reliably, however they also increase reasoning time and use up more tokens [3, p. 33-35]. Since Udengaard

already compared different model sizes in great detail in his report I consider it unnecessary for this study. An important thing to note is that Udengaard does not specify which *iteration* of GPT-5 he uses (GPT-5.3, GPT-5.4), and since the system defaults to whichever model iteration is newer, a 1:1 comparison is in theory impossible, as Udengaard could have been using a smaller and worse performing model for his tests, such as GPT-5.2 nano.

Dataset: I run all the tests on a locally compiled TPC-H dataset, an open-source synthetic dataset typically used to benchmark database systems. It creates data simulating a wholesale supplier company. I compile the dataset with a scale factor of 1, the same as Udengaard used, and load it into a PostgreSQL database [3, p.27]. Since the dataset is locally compiled, it is not the exact same as Udengaard's, however the two should be very similar, since both share a scale factor of 1.

Hardware: I run all the tests locally on a Surface Book 2 running WSL. I use an Intel i7-8650U @ 1.90GHz CPU. I do not expect hardware to have a significant impact on the results, as most of the time system's runtime comes from LLM reasoning. The tests were run over a 7-day period from Thursday 30/4-Thursday 7/5 2026.

4.2.2 Queries

For a more nuanced test I define three queries with varying levels of complexity, easy, medium and hard. The medium and hard queries are taken from Udengaard's own set of benchmark queries for the TPC-H dataset, available here: (GitHub). Note that the repository also contains a human-checked correct SQL script corresponding to each query, for reference. I run each query 50 times and average the runtime. The three queries I test with are:

Easy: *"Give me the first 10 phone numbers in the customers table."*
A simple query with little complexity to test baseline performance.

Medium: *"Create a plan that summarizes all line items shipped before a cutoff date, defined as July 1st 1998 minus 90 days. The query must group the results by return flag. For each group, calculate the total quantity, total extended price and average quantity. You don't need ordering or sorting."*
A more complex query with a filter and group by clause. This is the `0_0.txt` benchmark query, shortened to decrease complexity.

Hard: *"Find the five suppliers who generated the highest total revenue. Revenue is defined as* `l_extendedprice` $\times (1 -$ `l_discount`$)$ *aggregated over all lineitems for each supplier. Return the supplier information together with the computed revenue."*

A complicated query, asking the LLM to join and aggregate across several different tables. This is the `O_9.txt` benchmark query.

4.2.3 Variables

In addition to testing **framework** (MCP vs LangGraph) and **agent type** (single- or multi-agent) I am also testing **reasoning effort**, how deep the LLM thinks about a problem. For the single-agent system the Builder agent is set to low/high reasoning effort, and for the multi-agent system the reasoning effort of every agent is set to low/high. Reasoning effort is not a big part of Udengaard's original testing, and its impact on performance and accuracy is interesting to see.

Each configuration is tested across all three queries, each query being run 50 times over, resulting in 1200 total runs. With some complex runs taking over 20 minutes, testing is quite time-consuming. Table 1 shows every configuration that was tested. It should be noted that almost all runs on the harder queries fail due to the smaller LLM model used. The performance is still logged for these failed runs, as they fail only when receiving an error from the Wayang server, making their overall runtime equivalent to successful runs.

ID	Framework	Agent Type	Reasoning Effort
1	MCP	Single	Low
2	MCP	Single	High
3	MCP	Multi	Low
4	MCP	Multi	High
5	LangGraph	Single	Low
6	LangGraph	Single	High
7	LangGraph	Multi	Low
8	LangGraph	Multi	High

Table 1: Configurations Overview

4.3 Results

4.3.1 Performance

Although most of the runtime is likely spent on LLM reasoning or server latency on the side of the LLM provider, it is still interesting to see if any performance gains are made by switching away from a server-client model to a graph-based model. Figure 8 shows an overview of the performance across the different systems. Please note that the y-scale is not linear due to the wildly wide-ranging timescales on display. Overall, LangGraph seems to be

slightly more performant within easier queries, and with harder queries runtime performance seems to be a toss-up. We also see that going from low to high reasoning effort increases runtimes by roughly 4-6x.

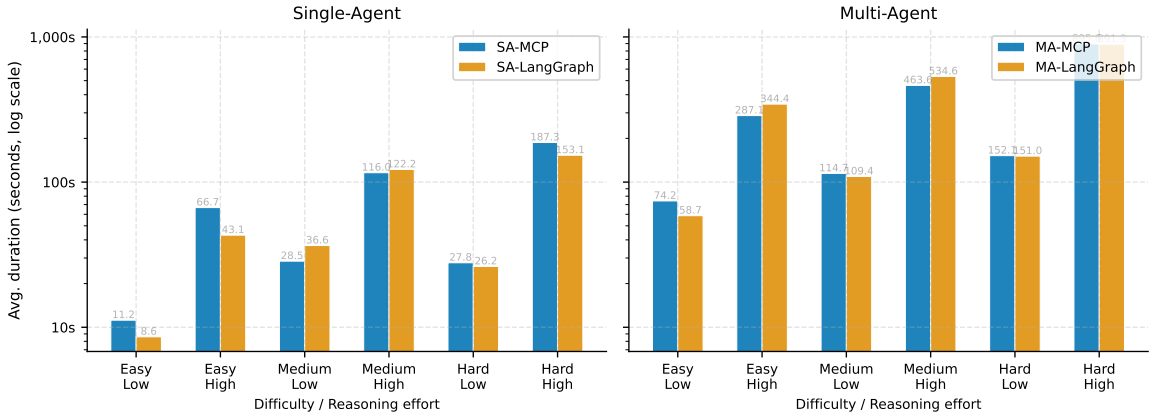


Figure 8: Runtime Performance Barplots

Doing visual inspections provide relatively weak conclusions, and I want to include some statistical correlation analysis to see if any statistically significant patterns emerge from the testing data. As mentioned, the testing data is aggregated across 50 runs, and for proper statistical correlation I first need to know what kind of distribution that data has.LLM-related tasks tends to be right-skewed, with a tail of occasional very slow runs. To check if this is also the case here, I include two scatter plots, shown in Figure 9. These are two randomly chosen configurations: an easy query for the single-agent pipeline and a hard query for the multi-agent system, both with low reasoning.

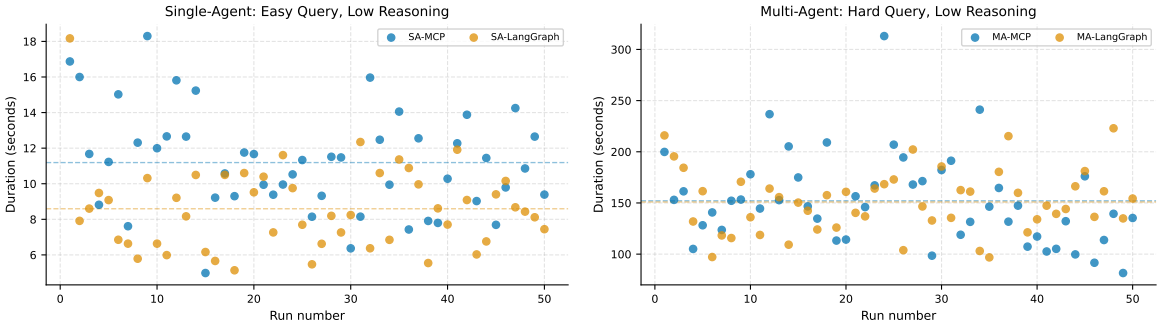


Figure 9: Scatterplots for Single- and Multi-Agent Systems

In the first scatterplot, MCP has a couple significant outliers in the $> 15s$ range, and in the second scatterplot a single major outlier is visible. Overall, the scatterplots suggest that some of the data is indeed right-skewed, with significant outliers for some configurations.

These outliers are generally to be expected when dealing with LLMs as server latency or reasoning spikes can introduce significant delay in the system.

Correlation Analysis The purpose of my tests is to compare the performance of the MCP system with the LangGraph system, and see if any differences are statistically significant. However, this right-skew makes it difficult to perform a standard t-test for statistical correlation, since that test assumes normal distribution. A chi-squared test is also not suitable for non-categorical data. Instead, I include a Mann-Whitney U test, which is an alternative test requiring independent groups and a continuous outcome. The outcome of my test is continuous runtime, and the groups are the two pipelines, MCP and LangGraph, that indeed operate independently of each other. The results are shown in Table 2 and Table 3.

Here, the p-value is defined as the probability of observing a difference at least as large as the one measured, assuming there is no real difference between the two. A lower p-value suggests a more statistically significant effect. The r-value is defined as the effect, where a positive r-value means that the MCP system is slower, and a negative r-value means the LangGraph system is slower. A larger r-value means the system is slower by a wider margin. I define an outcome as statistically significant if $p < 0.05$ and $r > \pm 0.3$. Significant results are marked in **bold**.

Configuration	Median MCP (s)	Median LG (s)	p-value	Effect r
Easy / Low	11.28	8.34	**<0.001**	**0.538**
Easy / High	59.4	40.46	**<0.001**	**0.783**
Medium / Low	27.4	34.54	**<0.001**	**-0.471**
Medium / High	109.14	118.98	0.123	-0.179
Hard / Low	27.11	26.2	0.581	0.064
Hard / High	190.68	148.91	**<0.001**	**0.548**

Table 2: Single-Agent Mann-Whitney U Test

Conclusion The test shows some statistically relevant differences in performance. For configurations with hard queries, as well as most medium queries, the differences are small, and the p-value insignificant. These configurations are mostly dominated by LLM reasoning time, so any potential framework overhead disappears into the noise. However, when considering configurations with the easy query, we can see a statistically relevant performance advantage to LangGraph. This suggests that the framework overhead, eg. runtime occupied by framework, is smaller on LangGraph, which is mainly visible on shorter

Configuration	Median MCP (s)	Median LG (s)	p-value	Effect r
Easy / Low	71.14	56.32	**<0.001**	**0.562**
Easy / High	294.97	332.15	**0.004**	**-0.338**
Medium / Low	104.13	103.09	0.785	0.032
Medium / High	433.86	504.1	**0.001**	**-0.372**
Hard / Low	146.53	148.85	0.728	-0.041
Hard / High	893.42	910.86	0.937	-0.010

Table 3: Multi-Agent Mann-Whitney U Test

runtimes. Overall, the choice of framework has a statistically significant effect on runtime, but only when the query is simple enough that the overhead is a meaningful fraction of total time. For medium and hard queries, where LLM reasoning occupies most of runtime, the two frameworks are mostly statistically indistinguishable.

4.3.2 Accuracy

The next metric is accuracy, eg. how many runs were successfully executed in a configuration, in percentage. Intuitively, I would not expect the new framework to have any measurable impact on accuracy, as logic is entirely conserved from the MCP system to the LangGraph one. In other words, seeing as the LLM is chiefly responsible for the accuracy of the system, and since we keep the same LLM logic intact across both implementations, accuracy should be generally equivalent across both MCP and LangGraph systems. Figure 10 shows an overview of the accuracy of all configurations. Surprisingly, we see that LangGraph is more accurate than MCP in all but one configuration.

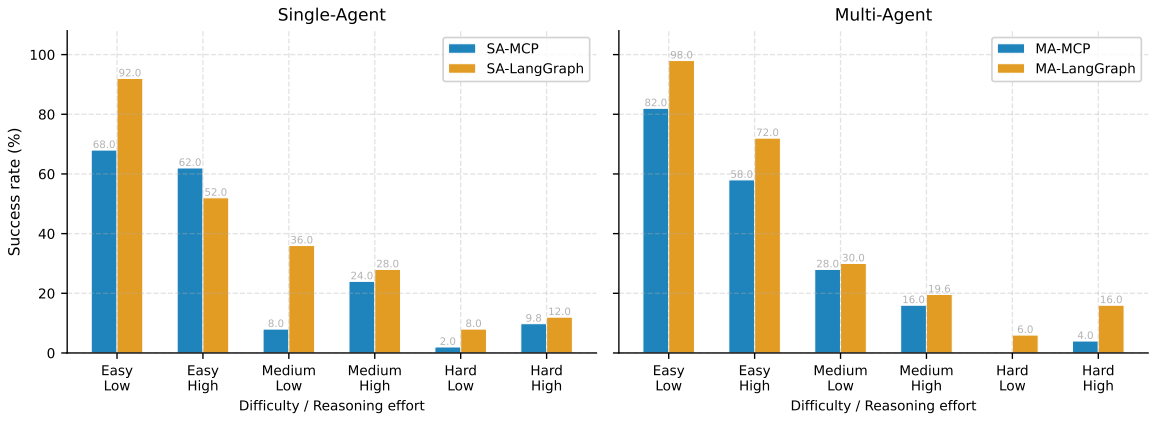


Figure 10: Success Rate Barplots

Correlation Analysis To judge whether these differences are statistically significant I use Fisher's Exact test. I could use a chi-squared test, but when expected counts are small (eg. $0/50$ or $2/50$ successful runs), the chi-squared approximation is inaccurate, and risk throwing off the p-value significantly. Fisher's Exact test is used to determine if there is a non-random relation between success and failure across the two systems. Table 4 and Table 5 shows the tests for each system.

Here, the p-value is defined the same way as before, and the odds-ratio o measures how much more likely one system is to produce a success, compared to the other system. An o of 1 means both systems are equally likely, an $o < 1$ means LangGraph has a higher likelihood of success and an $o > 1$ means MCP has a higher likelihood. I define an outcome as statistically significant if $p < 0.05$ and $o > 0.1$. Significant results are once again marked in **bold**.

Configuration	MCP (succ./fail.)	LG (succ./fail.)	Odds ratio	p-value
Easy / Low	34/16	46/4	**0.185**	**0.005**
Easy / High	31/19	26/24	1.506	0.419
Medium / Low	4/46	18/32	**0.155**	**0.001**
Medium / High	12/38	14/36	0.812	0.820
Hard / Low	1/49	4/46	0.235	0.362
Hard / High	5/46	6/44	0.797	0.760

Table 4: Single-Agent Fisher's Exact Test

Configuration	MCP (succ./fail.)	LG (succ./fail.)	Odds ratio	p-value
Easy / Low	41/9	49/1	**0.093**	**0.016**
Easy / High	29/21	36/14	0.537	0.208
Medium / Low	14/36	15/35	0.907	1.000
Medium / High	8/42	10/41	0.781	0.796
Hard / Low	0/50	3/47	0.000	0.242
Hard / High	2/48	8/42	0.219	0.092

Table 5: Multi-Agent Fisher's Exact Test

Conclusion The tests show that only 2 configurations are statistically significant. Although LangGraph was more accurate than MCP in all but one configuration, the majority of

these results do not meet the defined significance threshold. This implies that the differences in accuracy are likely coming down to random variations, rather than the choice of framework. The two configurations that do reach significance suggest that there may be specific conditions where the framework choice matters, but I don't find these observations to be sufficient to generalise across either system.

Regarding reasoning level we see that a higher level of reasoning actually has an adverse effect on many configurations, mostly in the easy and medium queries. I believe this might be due to the LLM 'overthinking' things on the easier queries, and producing overcomplicated results that do not execute. Overall, using high reasoning is not likely to be worth the 4-6x increase in time and tokens spent.

4.3.3 Lines of Code

Finally, a quite simple metric. I want to see if the new framework has any significant impact on the size of the codebase. Table 6 shows the amount of lines in each frameworks implementation.

	MCP	LangGraph
Single-Agent	231	**194**
Multi-Agent	374	**299**

Table 6: Comparison of Lines of Code

Overall, we do not see a large difference between the two frameworks. LangGraph is slightly less verbose than MCP, though at this scale, the difference is negligible. However, an important benefit of LangGraph is that it is arguably more clear and decoupled than MCP. For the MCP implementation, programmers would have to read the entire `query_wayang` tool to understand the dataflow, but with LangGraph logic is encapsulated within descriptive nodes, and edges make the structure and dataflow explicit. Additionally, data is centrally stored in the State as a typed dictionary, making it clearly visible at a glance.

Finally, decoupling is very important for future improvements to the pipeline. As the system is expanded, integrating new logic within a single, massive try-block becomes increasingly infeasible. The graph structure of LangGraph allows programmers to freely add and shift around nodes and make complicated routings with conditional edges, instead of working within branching if/else statements. As performance and accuracy gains seem to be marginal, this appears to be the main benefit of using LangGraph over MCP.

5 Related Work

Although I initially deemed LangGraph to be the most suitable candidate for comparison due to its simple structure and relative traction within the industry, I also identified several other potentially useful frameworks during this study. This section details some of these alternative frameworks for managing AI agents.

5.1 Universal Tool Calling Protocol (UTCP)

UTCP is an open-source, community-developed alternative to MCP servers [7]. Remember that the MCP framework follows a client-server model, where the LLM only sees the tool description and results - it does not manage the tool execution itself. In contrast to this, UTCP tries to be transport-agnostic, and allow for direct tool access. Tools are described in a schema manifest that details which tools are available, what parameters they take and how to invoke them directly (typically a HTTP URL and method type). When an LLM reads the manifest and decides to call a tool, the call is made directly to the tool's endpoint, and the response is fed back directly to the LLM. The tool still needs to live on a server to be reachable by HTTP, however. Overall, UTCP claims to be more lean and direct than MCP, cutting down on server-latency and code verbosity [8]. Its performance gains seem marginal for this use-case, but further testing would have be made to support this claim. In general, UTCP has much less industry traction than MCP, being developed by hobbyist individuals, instead of a larger tech company. It also currently only supports OpenAI API calls, severely limiting LLM vendor flexibility.

5.2 Microsoft Agent Framework (MAF)

MAF is an open-source framework designed for building agents and managing multi-agent workflows, built and maintained by Microsoft [9]. It consolidates Microsofts earlier efforts in the agentic AI framework space, notably AutoGen and Semantic Kernel. MAF was released after the initial research phase of this study, and was thus initially not considered for comparison. Like LangGraph, MAF models its workflow as a graph, meaning agents are defined as nodes and normal or conditional edges route one agent to another. It is unclear to me whether normal functional nodes are also allowed, or whether nodes can only be agentic. In general, MAF seems to define agents a bit differently than LangGraph, doing so declaratively with a nested system prompt within each node. MAF does provide some core features like observability, check-pointing, debugging and graph visualization that for LangGraph is mostly locked behind LangChain's commercial product, LangSmith. MAF seems to be quite robust and well-built, however the overall complexity of the framework also seems to be higher, with a larger focus on infrastructure.

6 Conclusion

This report investigated the use of an alternative agent framework in translating natural language queries to Wayang plans. To do this I refactored both of Alexander Udengaard's single- and multi-agent systems to use LangGraph instead of MCP servers. I found that the original MCP systems used a graph-like architecture to begin with, making the conversion to LangGraph quite straight-forward, and I was able to conserve all crucial logic in the refactoring process.

The new framework is then extensively evaluated across 8 different configurations, measuring performance and accuracy. The evaluations found that LangGraph was generally slightly more performant than MCP when runtimes were low and queries simple, but as runtimes and query complexity increased, LLM reasoning time took over, and buried much of the framework performance advantages. For medium to high complexity queries, neither framework performs faster than the other. I also found that accuracy is not significantly affected by changing to LangGraph, which aligned with my intuition that the framework layer should not affect accuracy.

Although substantial numerical gains in performance or accuracy were overall debatable, I found that the new framework allowed for a cleaner code structure, and I speculate that LangGraph's higher decoupling of logic allows the system to be easier to expand or improve upon further down the road.

Future Work and Limitations

This report used the TPC-H dataset for evaluation, a well-known, publicly available dataset. In his original paper, Udengaard suspects that the LLM the system uses was likely trained on information about the dataset, as he noticed variations in accuracy when running the pipelines on a private dataset compared to the public TPC-H set, implying a data leak. For this report to have more meaningful evaluations, tests would also have to be done on one or more private datasets, in addition to the TPC-H set, possibly with more than 50 iterations per configuration. One such test-setup could reasonably exclude low/high reasoning effort to cut down significantly on test time.

A similarly substantial comparison could also be made with some of the proposed alternative frameworks, UTCP or MAF. I believe UTCP's limitations (LLM vendor lock-in) make it unsuitable for long-term use, however a comprehensive comparison with MAF would

be interesting, as that framework also utilizes a graph-based workflow, and provides some features that for LangGraph are locked behind paid software.

On the use of AI

The abstract of this report was produced with the help of an AI, and AI was also used for checking of grammar and spelling. Additionally, I used some amount of Claude Code to help me convert the codebase from MCP to LangGraph. Beyond this, no AI was used.

Configuration	Mean MCP (s)	Mean LG (s)	t-stat	p-value	Cohen's d
Easy / Low	11.19	8.59	**4.974**	<**0.001**	**0.995**
Easy / High	66.74	43.1	**5.828**	<**0.001**	**1.166**
Medium / Low	28.53	36.56	**-3.751**	<**0.001**	**-0.750**
Medium / High	116.04	122.15	-0.494	0.623	-0.099
Hard / Low	27.81	26.24	1.247	0.215	0.249
Hard / High	187.34	153.09	**4.441**	<**0.001**	**0.884**

Table 7: Single-Agent Student's t-Test (Runtime Duration)

Configuration	Mean MCP (s)	Mean LG (s)	t-stat	p-value	Cohen's d
Easy / Low	74.16	58.69	**5.170**	**<0.001**	**1.034**
Easy / High	287.13	344.44	**-3.321**	**0.001**	**-0.664**
Medium / Low	114.73	109.36	0.858	0.393	0.172
Medium / High	463.6	534.65	**-2.440**	**0.017**	**-0.486**
Hard / Low	152.13	150.96	0.156	0.876	0.031
Hard / High	895.61	891.29	0.121	0.904	0.024

Table 8: Multi-Agent Student's t-Test (Runtime Duration)

Appendix

```
workflow = StateGraph(WayangAgentState)

workflow.add_node("build_plan", build_plan_node)
workflow.add_node("map_plan", map_plan_node)
workflow.add_node("validate_plan", validate_plan_node)
workflow.add_node("execute_plan", execute_plan_node)
workflow.add_node("debug_plan", debug_plan_node)
workflow.add_node("finish", finish_node)

workflow.set_entry_point("build_plan")
workflow.add_edge("build_plan", "map_plan")
workflow.add_edge("map_plan", "validate_plan")
workflow.add_conditional_edges("validate_plan", route_after_validate)
workflow.add_conditional_edges("execute_plan", route_after_execute)
workflow.add_edge("debug_plan", "map_plan")
workflow.add_edge("finish", END)

graph = workflow.compile()
```

Figure 11: Single-Agent LG Graph Code Snippet

```python
class WayangAgentState(TypedDict):
    # Input variables
    query: str
    model: str
    reasoning: Optional[str]
    use_debugger: bool

    # Working state variables
    raw_plan: Optional[WayangPlan]       # Output of Builder or Debugger
    wayang_plan: Optional[dict]          # Output of PlanMapper (executable JSON)
    val_success: bool
    val_errors: List[str]
    status_code: Optional[int]
    result: Optional[str]                # Raw response from Wyang server
    iteration: int                       # Debugger iteration counter

    # Instances
    debugger: Any                        # Debugger instance
    logger: Any                          # Logger instance
```

Figure 12: Single-Agent LG State Code Snippet

References

[1] "About apache wayang," Accessed: May 6, 2026. [Online]. Available: https://wayang.apache.org/docs/introduction/about/

[2] C. Stryker. "What is agentic ai?" Accessed: May 6, 2026. [Online]. Available: https://www.ibm.com/think/topics/agentic-ai

[3] A. H. Udengaard, "Agent-based systems for apache wayang plan generation," KISPECI-ISE programme, supervised by Zoi Kaoudi, Master Thesis, IT University of Copenhagen, Jan. 2026.

[4] "Model context protocol (mcp)," Accessed: May 6, 2026. [Online]. Available: https://github.com/modelcontextprotocol/modelcontextprotocol

[5] "Langgraph github readme," Accessed: May 6, 2026. [Online]. Available: https://github.com/langchain-ai/langgraph

[6] S. Banerjee. "Langchain vs. langgraph," Accessed: May 6, 2026. [Online]. Available: https://www.geeksforgeeks.org/artificial-intelligence/langchain-vs-langgraph/

[7] R. Radulescu. "Universal tool calling protocol (utcp)," Accessed: May 6, 2026. [Online]. Available: https://www.utcp.io/utcp-vs-mcp

[8] R. Radulescu. "Utcp vs mcp: A comprehensive comparison," Accessed: May 6, 2026. [Online]. Available: https://www.utcp.io/

[9] “Microsoft agent framework github readme,” Accessed: May 6, 2026. [Online]. Available: https://github.com/microsoft/agent-framework